COMP 489 Assignment 1 - Test Plan

Proxy Server, Basic Web Server and Client

Student: Boris Bojanov | ID: 3608550 | Course: COMP 489 Distributed Computing | Date executed: October 20, 2026

1. Objective and scope

This plan verifies the three programs required by the assignment: **SimpleWebServer** (basic HTTP server on port 9806), **ProxyServer** (threaded HTTP proxy on port 8888) and **SimpleClient** (asks for a resource URL, sends it *through the proxy* and prints the reply), plus the shared parser **MyHttp**. It covers compilation, correct delivery of HTML and JPEG files, the 400 and 404 status messages, request rewriting and two-way relaying in the proxy, concurrency (threads), user-friendly error handling in the client, robustness against bad input, and the instructor's named tests (testfile.html, testfile.jpg, http://127.0.0.1/..., https://www.google.com, ftp://ftp.us.dell.com/readme.txt).

2. Test environment and method

Method. Every test is executed by an automated runner (`tests/run_tests.py`) that compiles the sources, starts the web server and proxy as real processes, and talks to them over real TCP sockets. Each actual output in section 4 is copied from that run, not predicted. Files are checked byte-for-byte with SHA-256, and the web server's log is read to confirm what the proxy forwarded. Two small helper servers stand in for situations that are hard to produce on demand: one that replies *without* a Content-Length header (port 7779, like www.google.com) and one that accepts a connection and never answers (port 7777).

Platform. OpenJDK 21 (also compiled with `--release 11`), Linux; programs started from the project folder (the one containing `src` and `bin`). **Test files:** www/test/testfile.html (1,597 bytes), www/test/testfile.jpg (1,961,523 bytes), www/picture.JPEG (3,187,528 bytes), www/index.html. **To re-run:** `python tests/run_tests.py` from the COMP489-A1 folder. Tests marked *Run on PC* need internet access to www.google.com, which the test environment did not have; run `tests\run_internet_tests.ps1` on a PC with internet and record the result.

3. Summary of results

Area	Pass	Fail	Limitation	Run on PC	Total
Compile	2	0	0	0	2
Web server	18	0	0	0	18
Proxy	16	1	0	0	17
Client	11	1	0	0	12
Internet	0	0	1	2	3
All	**47**	**2**	**1**	**2**	**52**

47 of 52 tests passed. **2** failed (P14, CL9; see section 5). 1 is a documented design limitation (FTP), and 2 need an internet connection.

4. Test cases and results

Result key: **Pass** = actual matches expected; **Fail** = does not match (see section 5); **Limitation** = handled gracefully but outside the program's scope; **Run on PC** = needs internet access. In the Input column, \r\n marks the end of a line (CR LF) in a raw request.

4.1 Compile

ID	Test item	Input	Expected output	Actual output	Result / remarks
C1	Compile all four source files from one directory	`javac -d bin src/comp489/a1/*.java`	No compile errors; 4 classes produced	`exit code 0; 0 warning(s); classes: MyHttp.class, ProxyServer.class, SimpleClient.class, SimpleWebServer.class`	**Pass**
C2	Compile all four source files from one directory (Java 11 target)	`javac -d bin src/comp489/a1/*.java --release 11`	No compile errors; 4 classes produced	`exit code 0; 0 warning(s); classes: MyHttp.class, ProxyServer.class, SimpleClient.class, SimpleWebServer.class`	**Pass**

4.2 Web server

ID	Test item	Input	Expected output	Actual output	Result / remarks
W0	Start-up check: document root missing	`Start SimpleWebServer from a folder that has no src/comp489/a1/www`	Prints 'Document root is not a directory' and exits cleanly	`Document root: <tests>/src/comp489/a1/www | Error: Document root is not a directory.`	**Pass**
W1	Start-up with defaults	`java comp489.a1.SimpleWebServer (from project folder)`	Prints document root and 'listening on port 9806'	`Document root: <project>/src/comp489/a1/www | SimpleWebServer listening on port 9806 ... (Ctrl+C to stop)`	**Pass**
W2	Serve HTML file (testfile.html)	`GET /test/testfile.html HTTP/1.0\r\n\r\n`	HTTP/1.0 200 OK, Content-Type text/html, Content-Length 1597, body identical to file	`HTTP/1.0 200 OK; Content-Type text/html; Content-Length 1597; body 1597 bytes, identical=True`	**Pass**
W3	Serve JPEG file byte-for-byte (testfile.jpg)	`GET /test/testfile.jpg HTTP/1.0\r\n\r\n`	200 OK, image/jpeg, 1961523 bytes, SHA-256 identical to file on disk	`HTTP/1.0 200 OK; image/jpeg; 1961523 bytes; sha256 1004d5260e6d7151 vs file 1004d5260e6d7151`	**Pass**
W4	Serve large binary (3.1 MB picture.JPEG)	`GET /picture.JPEG HTTP/1.0\r\n\r\n`	200 OK, 3187528 bytes, identical to file	`HTTP/1.0 200 OK; Content-Type image/jpeg; 3187528 bytes; identical=True`	**Pass**
W5	Default document for '/'	`GET / HTTP/1.0\r\n\r\n`	200 OK with the contents of www/index.html	`HTTP/1.0 200 OK; Content-Length 0; body 0 bytes`	**Pass** index.html is currently empty (0 bytes); add content before submitting
W6	Query string is ignored	`GET /test/testfile.html?lang=en HTTP/1.0\r\n\r\n`	200 OK, same 1597-byte file	`HTTP/1.0 200 OK; body 1597 bytes, identical=True`	**Pass**
W7	Missing file returns 404	`GET /test/nope.html HTTP/1.0\r\n\r\n`	HTTP/1.0 404 Not Found with a short message	`HTTP/1.0 404 Not Found; body: File not found: <project>/src/comp489/a1/www/test/nope.html`	**Pass** Message reveals the server's full disk path; consider 'File not found: /test/nope.html'
W8	Directory requested without trailing '/'	`GET /test HTTP/1.0\r\n\r\n`	404 Not Found (a directory is not a file)	`HTTP/1.0 404 Not Found`	**Pass**
W9	Non-GET method rejected (POST)	`POST /test/testfile.html HTTP/1.0\r\nContent-Length: 0\r\n\r\n`	HTTP/1.0 400 Bad Request	`HTTP/1.0 400 Bad Request; body: Method 'POST' not supported. Only GET is allowed.`	**Pass**

ID	Test item	Input	Expected output	Actual output	Result / remarks
W10	Non-GET method rejected (HEAD)	`HEAD / HTTP/1.0\r\n\r\n`	HTTP/1.0 400 Bad Request	`HTTP/1.0 400 Bad Request`	**Pass**
W11	Path traversal with '..' blocked	`GET /../SimpleWebServer.java HTTP/1.0\r\n\r\n`	404 (Forbidden); source code NOT returned	`HTTP/1.0 404 Not Found; body: Forbidden`	**Pass**
W12	URL-encoded traversal (%2e%2e) blocked	`GET /%2e%2e/%2e%2e/%2e%2e/%2e%2e/README.md HTTP/1.0\r\n\r\n`	404 (Forbidden); README.md NOT returned	`HTTP/1.0 404 Not Found; body: Forbidden`	**Pass**
W13	Absolute-path escape ('//etc/passwd') blocked	`GET //etc/passwd HTTP/1.0\r\n\r\n`	404 (Forbidden); system file NOT returned	`HTTP/1.0 404 Not Found; body: Forbidden`	**Pass** Windows equivalent: GET /C:/Windows/win.ini
W14	Malformed percent-escape	`GET /%zz HTTP/1.0\r\n\r\n`	HTTP/1.0 400 Bad Request; server keeps running	`HTTP/1.0 400 Bad Request; body: Bad Request: Invalid or malformed URL path.`	**Pass**
W15	Server survives a client that connects and sends nothing	`Open a TCP connection, close it, then GET /test/testfile.html`	No crash; next request still gets 200 OK	`HTTP/1.0 200 OK on the follow-up request`	**Pass**
W16	Concurrent clients (one thread per connection)	`10 simultaneous GET /test/testfile.jpg`	All 10 receive the complete, identical image	`10/10 identical in 0.09s`	**Pass**
W17	Command-line arguments: port and document root	`java SimpleWebServer 9807 <path>/www/test ; GET /testfile.html`	Listens on 9807 and serves from the given folder	`HTTP/1.0 200 OK; body identical=True`	**Pass**

4.3 Proxy

ID	Test item	Input	Expected output	Actual output	Result / remarks
P1	Start-up with defaults	`java comp489.a1.ProxyServer`	Prints 'listening on port 8888'	`ProxyServer listening on port 8888 ... (Ctrl+C to stop)`	**Pass**
P2	Forward absolute-URL request (HTML)	`GET http://localhost:9806/test/testfile.html HTTP/1.0\r\nHost: localhost:9806\r\n\r\n`	200 OK and the identical 1597-byte file relayed back	`HTTP/1.0 200 OK; 1597 bytes; identical=True`	**Pass**
P3	Request line is rewritten for the web server	`(same request as P2)`	Web server log shows 'GET /test/testfile.html HTTP/1.0' (path only, no http://)	`request: GET /test/testfile.html HTTP/1.0`	**Pass**
P4	Forward binary (JPEG) byte-for-byte	`GET http://localhost:9806/test/testfile.jpg HTTP/1.0\r\n\r\n`	1961523 bytes, SHA-256 identical to file	`HTTP/1.0 200 OK; 1961523 bytes; sha256 1004d5260e6d7151 vs 1004d5260e6d7151`	**Pass**
P5	Instructor form: http://127.0.0.1/...	`GET http://127.0.0.1:9806/test/testfile.html HTTP/1.0\r\n\r\n`	200 OK, identical file (127.0.0.1 is allowed)	`HTTP/1.0 200 OK; identical=True`	**Pass** The spec's http://127.0.0.1/testfile.html uses port 80; our server runs on 9806
P6	Origin-form request + Host header	`GET /test/testfile.html HTTP/1.0\r\nHost: localhost:9806\r\n\r\n`	Proxy takes host/port from Host: header; 200 OK	`HTTP/1.0 200 OK; identical=True`	**Pass**
P7	404 relayed from web server	`GET http://localhost:9806/missing.html HTTP/1.0\r\n\r\n`	HTTP/1.0 404 Not Found relayed unchanged	`HTTP/1.0 404 Not Found`	**Pass**
P8	400 relayed for non-GET	`POST http://localhost:9806/test/testfile.html HTTP/1.0\r\nContent-Length: 5\r\n\r\nhello`	HTTP/1.0 400 Bad Request relayed (proxy forwards the body using Content-Length)	`HTTP/1.0 400 Bad Request`	**Pass**

ID	Test item	Input	Expected output	Actual output	Result / remarks
P9	Response with no Content-Length (reads to end of stream)	`GET http://localhost:7779/x HTTP/1.0\r\n\r\n`	Whole body relayed; connection then closed (no hang)	`HTTP/1.0 200 OK; body b'No Content-Length here.\nSecond line.\n'`	**Pass** Test server on port 7779 that sends no Content-Length, like www.google.com
P10	Client keeps its side open after the request (half-close test)	`GET http://localhost:9806/test/testfile.html HTTP/1.0\r\n\r\n (client never half-closes)`	Full response delivered, then proxy closes the connection without waiting	`HTTP/1.0 200 OK; identical=True; finished in 0.40s`	**Pass**
P11	Host not on the allow-list is refused	`GET http://example.org/ HTTP/1.0\r\n\r\n`	Proxy logs 'Access denied to host: example.org' and closes the connection	`client received 0 bytes; proxy log: Access denied to host: example.org`	**Pass** Connection is closed without an HTTP error reply
P12	Invalid port rejected	`GET http://localhost:99999/ HTTP/1.0\r\n\r\n`	Proxy logs 'Invalid target host or port' and closes the connection	`client received 0 bytes`	**Pass**
P13	Target web server not running	`GET http://localhost:9899/ HTTP/1.0\r\n\r\n`	Connection closed (or an error reply); proxy keeps running	`client received 0 bytes; proxy log: Proxy relay ended: Connection refused`	**Pass** Closed without an HTTP error reply (e.g. 502 Bad Gateway)
P14	Garbage request line	`HELLO\r\n\r\n`	Connection closed; proxy keeps running	`No reply; connection still open after 10 s (proxy thread blocked waiting for the client)`	**Fail**
P15	Concurrent clients through the proxy (threads)	`10 simultaneous JPEG requests via the proxy`	All 10 complete and identical	`10/10 identical in 0.11s`	**Pass**
P16	Proxy still healthy after all error cases	`GET testfile.html after P11-P15`	200 OK	`HTTP/1.0 200 OK`	**Pass**
P17	Command-line argument: listen port	`java ProxyServer 8889 ; SimpleClient localhost 8889`	Proxy listens on 8889; client gets 200 OK	`HTTP/1.0 200 OK`	**Pass**

4.4 Client

ID	Test item	Input	Expected output	Actual output	Result / remarks
CL1	Fetch HTML via proxy and print it	`http://localhost:9806/test/testfile.html`	Prints status line, headers, then the HTML text	`HTTP/1.0 200 OK | Content-Type: text/html | Content-Length: 1597 ... HTML body printed=True`	**Pass**
CL2	Fetch JPEG via proxy	`http://localhost:9806/test/testfile.jpg`	200 OK, image/jpeg, Content-Length 1961523; raw bytes printed	`HTTP/1.0 200 OK | Content-Type: image/jpeg | Content-Length: 1961523; all 1961523 image bytes printed=True`	**Pass** Spec: client only prints what it receives; binary shows as unreadable characters
CL3	Missing resource	`http://localhost:9806/test/nope.html`	Prints 'HTTP/1.0 404 Not Found' and the message	`HTTP/1.0 404 Not Found`	**Pass**
CL4	Blank input uses the default URL	`(press Enter)`	Uses http://localhost:9806/ and prints the response	`No URL entered, using default: http://localhost:9806/ | HTTP/1.0 200 OK`	**Pass**
CL5	Bare path rejected	`/index.html`	Asks for a full URL	`Please enter a full URL, e.g. http://localhost:9806/`	**Pass**
CL6	Malformed URL rejected	`http://bad url`	'Not a valid URL' message, no crash	`Not a valid URL: Illegal character in authority at index 10: http://bad url`	**Pass**
CL7	Unsupported scheme (ftp://)	`ftp://ftp.us.dell.com/readme.txt`	Explains only http/https are supported	`Unsupported scheme 'ftp'. Use http:// or https://`	**Pass** FTP is not HTTP; this proxy forwards HTTP only (see I2)

ID	Test item	Input	Expected output	Actual output	Result / remarks
CL8	Response with no Content-Length	`http://localhost:7779/x`	Whole body printed; client exits normally	`HTTP/1.0 200 OK | Content-Type: text/plain | No Content-Length here. | Second line.; took 0.1s`	**Pass**
CL9	Host not allowed by proxy	`http://example.org/`	Clear message that no response was received	`'null 0 null'`	**Fail**
CL10	Proxy not running	`http://localhost:9806/ [args: localhost 8899]`	'Client error: Connection refused (is the proxy running ...?)'	`Client error: Connection refused (is the proxy running on localhost:8899?)`	**Pass**
CL11	Proxy accepts but never answers (SO_TIMEOUT)	`http://localhost:9806/ [args: localhost 7777]`	Timeout message after about 7 s, no hang	`Timed out: no response from localhost:7777 (connect limit 5000 ms, read limit 7000 ms). (after 7.1s)`	**Pass**
CL12	URL with query string	`http://localhost:9806/test/testfile.html?x=1&y=2`	200 OK, HTML printed	`HTTP/1.0 200 OK`	**Pass**

4.5 Internet

ID	Test item	Input	Expected output	Actual output	Result / remarks
I1	Instructor test: https://www.google.com	`https://www.google.com`	Proxy contacts www.google.com:443 and the client reports the outcome clearly	`Not run here: the test environment cannot reach www.google.com`	**Run on PC** Run run_internet_tests.ps1 on your PC and fill in the actual result
I2	Instructor test: ftp://ftp.us.dell.com/readme.txt	`ftp://ftp.us.dell.com/readme.txt`	Handled gracefully (FTP is a different protocol from HTTP)	`Unsupported scheme 'ftp'. Use http:// or https://`	**Limitation** Client rejects ftp:// before contacting the proxy; the proxy forwards HTTP only. No crash.
I3	Instructor test: http://www.google.com/	`http://www.google.com/`	200 OK (or 301) and Google's HTML printed; no Content-Length, read to end of stream	`Not run here: the test environment cannot reach www.google.com`	**Run on PC** Run run_internet_tests.ps1 on your PC and fill in the actual result

5. Failures, limitations and follow-up

P14 - Proxy hangs on a malformed request line.

Cause: For 'HELLO', MyHttp.parseStartLine sets neither method nor protocol, so MyHttp.parseBody treats the message as a *response* with no Content-Length and calls readAllBytes() on the client socket. That blocks until the client disconnects, tying up a proxy thread.

Fix: Record in parseStartLine whether the start line was a response (it begins with 'HTTP/'), and read to end of stream only for responses. Re-run P14.

CL9 - Client prints 'null 0 null' when the proxy closes without replying.

Cause: When the proxy refuses a host (or cannot reach the target) it closes the connection without an HTTP reply. SimpleClient then prints the empty fields of MyHttp.

Fix: In SimpleClient, if response.protocol is null print 'No response received: the proxy closed the connection (the host may be blocked or the server unreachable).' Optionally have the proxy send '403 Forbidden' for blocked hosts and '502 Bad Gateway' when the target is unreachable. Re-run CL9, P11, P13.

I2 - FTP (limitation). ftp:// is a different protocol from HTTP; SimpleClient explains that only http:// and https:// are supported instead of failing. Supporting it would need an FTP client in the proxy.

I1 - https://www.google.com (run on PC). The proxy forwards plain HTTP. For an https:// URL it connects to port 443, where Google expects an encrypted TLS connection, so a normal page is *not* expected; the test checks that the client reports the outcome clearly and nothing hangs. Full HTTPS support would need CONNECT tunnelling.

I3 - http://www.google.com/ (run on PC). Google replies without a Content-Length header; the same code path passed locally in P9 and CL8.

Other observations (tests pass): W5 - www/index.html is empty, so '/' returns 200 with no content; W7 - the 404 message includes the server's full disk path, which is better left out.

6. Requirement coverage

Requirement (from the assignment)	Covered by
Three programs: client, basic web server, proxy server	C1, C2, W1, P1, CL1
Web server returns the file's data (HTML and JPEG) to the socket	W2, W3, W4, W5, W6
Web server returns status messages (404 Not Found, 400 Bad Request)	W7, W8, W9, W10, W14
Web server is safe and robust (no files outside the document root, survives bad input)	W11, W12, W13, W14, W15
Proxy listens on one socket and forwards on another; relays the reply	P2, P3, P4, P6, P7, P8, P9
Proxy uses threads to send and receive simultaneously	P10, P15, W16
Client: user inputs the resource URL; request goes via the proxy; reply printed	CL1, CL2, CL3, CL4, CL12
User-friendly error handling	CL5, CL6, CL7, CL9, CL10, CL11, P11-P14, W0
Instructor's tests: testfile.html / testfile.jpg / 127.0.0.1 / google / ftp	W2, W3, P5, I1, I2, I3
Programs compile from one working directory	C1, C2